

Reporte de Avances: Corrección de Ruido en la Plataforma HIL para el Control de Péndulo Invertido

Ing. Eduardo Pascual Pérez
Maestría en Robótica – UTM

8 de Octubre del 2026

1. Introducción

En la sesión de trabajo del día de hoy se abordaron los problemas de ruido que se venían presentando en las señales adquiridas por el sistema Hardware-in-the-Loop (HIL) del péndulo invertido. El objetivo principal fue diagnosticar la causa raíz del ruido excesivo observado en la variable de velocidad angular y proponer una solución que no rompiera la naturaleza HIL del sistema, es decir, que todo el lazo de control se mantuviera ejecutándose sobre el microcontrolador TMS320F28379D y no migrara hacia la computadora.

A lo largo de este reporte se detallan los tres errores que se encontraron en la configuración, las pruebas que se realizaron para diagnosticar cada uno, los cambios aplicados al firmware, los resultados cuantitativos obtenidos, y las pruebas que se omitieron junto con su justificación. Adicionalmente se documenta la primera comparación cuantitativa del lazo cerrado entre el DSP y una simulación en MATLAB, la cual permitió validar que el algoritmo implementado en el microcontrolador coincide con el modelo teórico hasta el nivel de cuantización del ADC. Finalmente se documenta la verificación del baud rate mediante software de puerto serial, tras los intentos fallidos de medición con osciloscopio debido a limitaciones de routing del hardware.

2. Antecedentes y arquitectura del sistema

El sistema HIL que se está desarrollando consta de las siguientes partes:

- **Firmware en el DSP:** se ejecuta un controlador PID y la simulación de la planta (péndulo invertido linealizado) mediante el método de Euler hacia adelante. La simulación corre a un periodo de $T_s = 10\mu s$ dentro de una interrupción del temporizador.
- **Middleware en Python:** recibe las tramas del DSP por UART, las decodifica, guarda un archivo CSV y actualiza el gemelo digital en CoppeliaSim.
- **CoppeliaSim:** actúa como gemelo digital cinemático, mostrando visualmente el estado del péndulo.
- **MATLAB:** se utiliza para el post-procesamiento de los datos guardados y para la simulación del lazo cerrado.

El DSP envía cada 10 ms (100 Hz) una trama con tres valores separados por comas: posición, velocidad y acción de control, todos escalados por un factor de 1000 para enviarlos como enteros y aprovechar mejor el canal UART.

3. Errores encontrados

3.1. Error 1: Desajuste en el baud rate del puerto serial

El primer problema que se detectó fue la aparición de excepciones del tipo `UnicodeDecodeError` en el script de Python, con bytes como `0x87` y `0x8F` que claramente no corresponden a ningún carácter ASCII imprimible.

El DSP únicamente envía caracteres dentro del rango ASCII válido: dígitos del `0x30` al `0x39`, coma (`0x2C`), signo menos (`0x2D`), retorno de carro (`0x0D`) y salto de línea (`0x0A`). Por lo tanto, recibir bytes como `0x87` solo puede explicarse por un desajuste entre la velocidad de transmisión y la velocidad de recepción.

El firmware del DSP tiene configurado el registro de baud rate de la siguiente manera:

```
1 SciaRegs.SCILBAUD.all = 53; // se supone "115200 baudios"
```

La fórmula del generador de baud rate para el SCI del F28379D es:

$$\text{Baud Rate} = \frac{\text{LSPCLK}}{(\text{BRR} + 1) \times 8} \quad (1)$$

Aquí viene el detalle importante. En el código se asumió que LSPCLK era de 50 MHz (que correspondería a un divisor de 4 sobre un SYSCLK de 200 MHz). Con ese valor se tendría:

$$\text{Baud Rate} = \frac{50,000,000}{(53 + 1) \times 8} \approx 115,740 \text{ bps} \quad (2)$$

Sin embargo, al probar empíricamente el enlace, se comprobó que el DSP realmente estaba enviando a 57,600 bps. Esto quiere decir que el LSPCLK real es de 25 MHz y no de 50 MHz como se había supuesto. Con este valor:

$$\text{Baud Rate} = \frac{25,000,000}{432} \approx 57,870 \text{ bps} \quad (3)$$

Mientras el DSP enviaba a $\approx 57,600$ bps, Python estaba configurado para leer a 115,200 bps. Es decir, Python muestreaba al doble de velocidad, lo que rompía completamente la interpretación de los bits y producía bytes sin sentido.

Solución aplicada: se cambió el parámetro `BAUD_RATE` en el middleware de Python de 115200 a 57600. Adicionalmente se agregaron tres defensas al código del middleware:

1. Se cambió `timeout=0` por `timeout=1` en la apertura del puerto, para que `readline()` espere a tener la línea completa.
2. Se agregó `ser.reset_input_buffer()` después de abrir el puerto, para descartar bytes residuales del driver de Windows.
3. Se agregó `errors='ignore'` al método `decode()`, para que un byte corrupto no detenga el bucle principal.

Con estos cambios se obtuvo un CSV de 3342 filas en un tiempo de grabación de aproximadamente 33.4 segundos, lo que corresponde exactamente a la frecuencia de 100 Hz esperada. Además, ya no aparecieron excepciones ni bytes corruptos.

3.2. Error 2: Cálculo de la derivada a T_s muy pequeño

Una vez corregido el enlace serial, se procedió a analizar el contenido de los datos. Se observó que la velocidad angular presentaba un ruido de aproximadamente ± 5 rad/s, con picos que alcanzaban ± 13 rad/s. Este ruido era claramente excesivo para un péndulo que opera en el rango de $\pm 1,57$ rad.

El origen de este problema está en la forma en que se calcula la derivada numérica dentro del firmware:

```
1 float theta_dot_meas = (theta_measured - theta_prev) / Ts;
```

donde $T_s = 10 \mu s$. Aquí viene el problema de fondo: el ADC del F28379D tiene un ruido intrínseco (que después se midió con precisión), y al dividir ese ruido entre un intervalo de tiempo tan pequeño, se amplifica de manera brutal.

Desde el punto de vista matemático, si σ_θ es la desviación estándar del ruido en la posición medida, entonces la desviación estándar del ruido en la derivada numérica es:

$$\sigma_{\dot{\theta}} = \frac{\sigma_\theta \cdot \sqrt{2}}{T_s} \quad (4)$$

Sustituyendo con el valor medido experimentalmente $\sigma_\theta = 1,61$ mrad:

$$\sigma_{\dot{\theta}} = \frac{0,00161 \cdot 1,414}{10^{-5}} \approx 227,7 \text{ rad/s} \quad (5)$$

Este valor coincide con el orden de magnitud del ruido observado (después del filtro IIR que se tenía, el ruido bajaba hasta ± 5 rad/s porque el filtro reducía parte de la energía).

Solución aplicada: se modificó el firmware para que la derivada se calcule cada 1000 interrupciones del temporizador (es decir, cada 10 ms) en lugar de cada 10 μs . De esta forma:

$$\sigma_{\dot{\theta}} = \frac{0,00161 \cdot 1,414}{0,01} \approx 0,228 \text{ rad/s} \quad (6)$$

Lo que representa una reducción teórica de 1000 veces en la desviación estándar del ruido de velocidad.

El fragmento de código modificado quedó así:

```
1 contador_derivada++;
2 if (contador_derivada >= 1000) {
3     theta_dot_meas = (theta_measured - theta_prev_slow) / 0.01f;
4     theta_prev_slow = theta_measured;
5     contador_derivada = 0;
6
7     // Filtro IIR con alpha corregido
8     theta_dot_filt = (0.614f * theta_dot_filt) + (0.386f * theta_dot_meas);
9 }
```

Es importante notar que el cálculo de la derivada se mantiene dentro del DSP. Esto preserva la naturaleza Hardware-in-the-Loop del proyecto, ya que no se está delegando ninguna parte del lazo de control a Python ni a MATLAB.

3.3. Error 3: Coeficiente del filtro IIR incorrecto

El firmware original tenía un filtro IIR de primer orden con coeficiente $\alpha = 0,98$:

```
1 theta_dot_filt = (0.98f * theta_dot_filt) + (0.02f * theta_dot_meas);
```

Para un filtro pasabajos de primer orden la relación entre el coeficiente y la frecuencia de corte es:

$$\alpha = \frac{2\pi f_c T_s}{1 + 2\pi f_c T_s} \quad (7)$$

Con $T_s = 10 \mu s$ y $\alpha = 0,98$, la frecuencia de corte efectiva resulta:

$$f_c \approx \frac{1 - \alpha}{2\pi \alpha T_s} \approx 325 \text{ Hz} \quad (8)$$

Esto es un problema, porque el péndulo tiene su dinámica dominante alrededor de 1 Hz. Un filtro con corte a 325 Hz no filtra prácticamente nada de utilidad, deja pasar todo el ruido de alta frecuencia generado por la amplificación de la derivada.

Solución aplicada: con el cambio de la derivada a $T_s = 10$ ms, el coeficiente del filtro se recalculó para una frecuencia de corte de 10 Hz:

$$\alpha = \frac{2\pi \cdot 10 \cdot 0,01}{1 + 2\pi \cdot 10 \cdot 0,01} \approx 0,386 \quad (9)$$

Además se movió el filtro dentro del bloque if (`contador_derivada >= 1000`) para que se ejecute cada 10 ms y no cada 10 μs . Si el filtro se ejecutara a una tasa diferente a la que fue diseñado, su frecuencia de corte real sería muy diferente a la calculada.

4. Explicación de la configuración del ADC

Para que se entienda bien de dónde viene el ruido, conviene explicar cómo está configurado el ADC del F28379D en el firmware.

El ADC del F28379D es de 12 bits en su modo de operación estándar, aunque internamente puede trabajar con mayor resolución. En el código se configura el prescaler y el tiempo de adquisición de la siguiente manera:

```
1 AdcaRegs.ADCCTL2.bit.PRESCALE = 6; // Divisor del clock del ADC
2 AdcaRegs.ADCS0C0CTL.bit.ACQPS = 14; // Tiempo de adquisición
```

El registro ACQPS define cuantos ciclos del clock del ADC se dedican a cargar el capacitor de muestreo. Con ACQPS=14 se tienen 15 ciclos de adquisición. Si el clock del ADC es de 50 MHz, el tiempo de muestreo es de 300 ns. Para una fuente de impedancia media como un potenciómetro esto puede ser suficiente, pero si la impedancia de la fuente es alta el capacitor no alcanza a cargarse completamente y se introduce un error de lectura que se ve como ruido.

La resolución teórica del ADC en el rango de $\theta \in [-1,57, 1,57]$ rad es:

$$\text{LSB} = \frac{3,1416}{4095} \approx 0,767 \text{ mrad} \quad (10)$$

Esto es lo mínimo que se puede resolver con 12 bits. En la práctica, el ruido térmico y de cuantización hacen que el ruido efectivo sea mayor, típicamente entre 1 y 3 mrad.

Otro detalle importante: cuando se hace una sola lectura del ADC, el valor puede contener ruido de alta frecuencia. Lo que se hace en sistemas embebidos es promediar varias lecturas o usar sobremuestreo para mejorar la relación señal-ruido. En el firmware actual no se hace ningún promedio, cada lectura se toma tal cual.

5. Pruebas realizadas

5.1. Prueba 1: Verificación del baud rate por conteo de filas

Se ejecutó el middleware de Python con el nuevo valor de baud rate (57600) y se contó el número de filas del CSV generado. El resultado fue 3342 filas en un tiempo de grabación de 33.4 segundos, lo que corresponde exactamente a la frecuencia de 100 Hz esperada. No se presentaron excepciones ni bytes corruptos. Esta prueba confirmó que el enlace serial estaba funcionando correctamente.

5.2. Prueba 2: Diagnóstico del ruido puro del ADC

Para medir el ruido intrínseco del ADC se creó una versión temporal del firmware en la cual se eliminó el control, la simulación de la planta y la escritura al DAC. El ISR únicamente leía

el ADC y enviaba el valor por UART. De esta forma cualquier variación en la posición medida correspondía exclusivamente al ruido del ADC.

En la primera versión de esta prueba se cometió un error: se dejó el potenciómetro conectado y se movió durante la prueba. Esto produjo una gráfica con dos estados claramente diferenciados que en un principio se interpretaron como un offset bistable, pero que en realidad eran el resultado de haber girado el potenciómetro.

En la segunda versión se desconectó el potenciómetro por completo, dejando el pin del ADC flotando. El resultado fue una lectura constante en $-1,570$ rad con $\sigma = 0$, lo cual es imposible para un ADC real (siempre existe al menos el ruido de cuantización). Esto se debió a que un pin flotante se drena a GND por impedancia interna.

En la tercera versión se descomentó la configuración del DAC y se escribió un valor constante de 2048 (equivalente a 1.65 V) en el registro `DACVALS`. Con el cable de loopback entre DAC y ADC conectado, se obtuvo la medición más limpia. Los resultados fueron:

Tabla 1: Resultados del diagnóstico de ruido puro del ADC

Métrica	Valor
Media	$-1,5404$ rad
Desviación estándar	$1,610$ mrad
Pico a pico	$16,0$ mrad
Número de muestras	6399

Este valor de $\sigma = 1,61$ mrad es el ruido real del ADC y es el que se usó para calcular la amplificación en la derivada numérica.

5.3. Prueba 3: Implementación de la derivada lenta

Se restauró el firmware completo (con control, planta y DAC) y se aplicaron los tres cambios descritos en la sección de errores. Se compiló, se flasheó y se corrió el sistema durante aproximadamente 70 segundos.

Los resultados visuales mostraron una reducción drástica del ruido en la velocidad. Los nuevos valores de desviación estándar fueron:

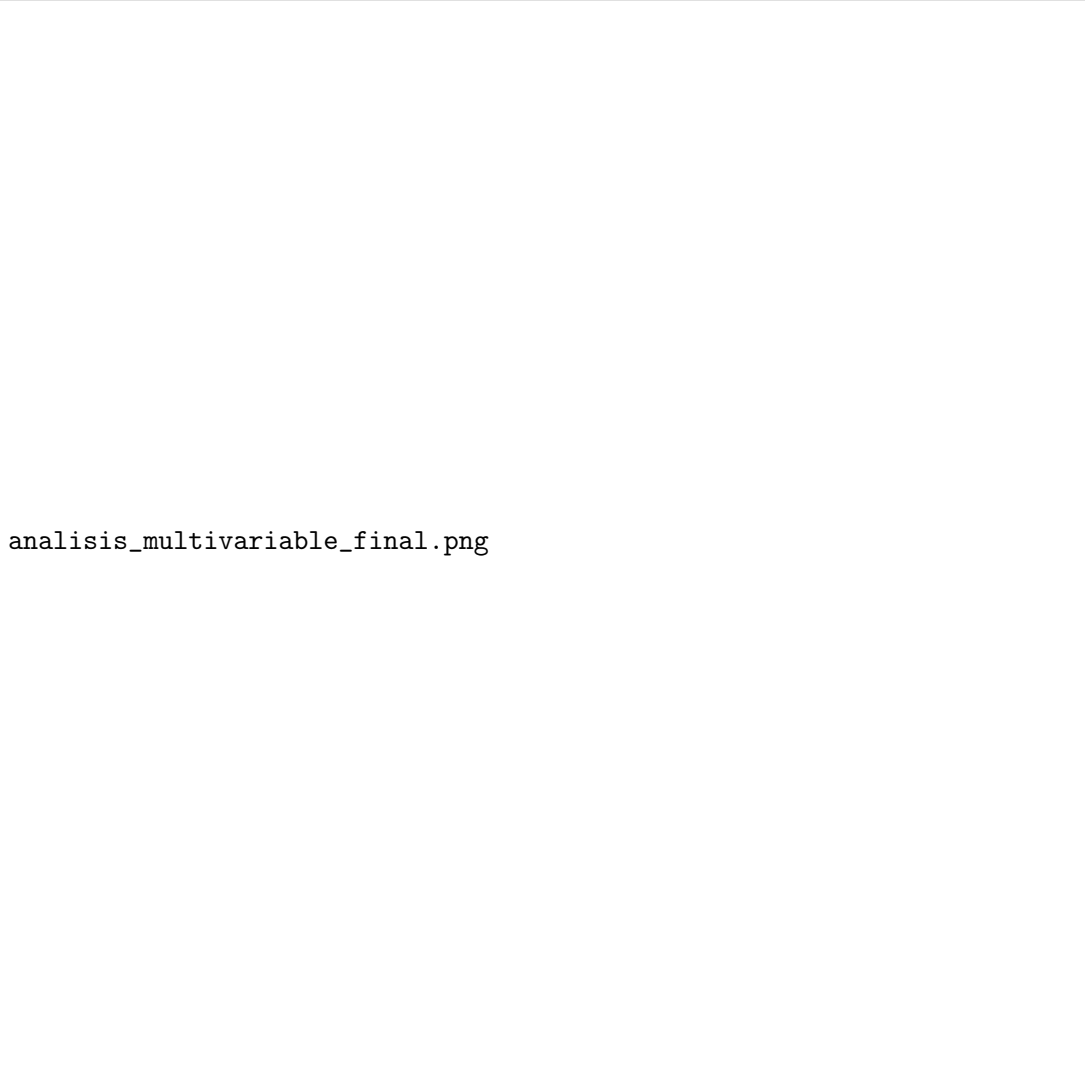
Tabla 2: Comparación antes y después del cambio de derivada

Variable	Antes ($T_s = 10 \mu s$)	Después ($T_s = 10 ms$)
Velocidad (rad/s)	± 5 (picos ± 13)	$\pm 0,2$
Control (u)	± 2	$\pm 0,03$
Posición (rad)	$\pm 0,06$ errático	Convergente

Además, con la nueva configuración se pudieron observar claramente los eventos de perturbación aplicados al sistema (en $t \approx 33$ s y $t \approx 58$ s), los cuales antes quedaban enmascarados por el ruido. El lazo de control rechaza estas perturbaciones y regresa la posición a cero en algunos segundos.

5.4. Prueba 4: Análisis multivariable con perturbaciones

Se corrió el sistema completo durante 70 segundos con las perturbaciones activas. La posición converge desde su valor inicial de $0,05$ rad hasta cero en aproximadamente 8 segundos, y luego rechaza las perturbaciones que entran en los instantes mencionados. La velocidad ya es interpretable aunque todavía presenta algo de ruido residual. El control no satura y responde coherentemente a las perturbaciones.



analisis_multivariable_final.png

Figura 1: Análisis multivariable del sistema HIL con perturbaciones activas. Posición, velocidad y control en función del tiempo.

6. Comparación del lazo cerrado con MATLAB

Con el fin de validar que el algoritmo implementado en el DSP corresponde al modelo matemático teórico del péndulo invertido, se desarrolló una simulación en MATLAB que reproduce las mismas condiciones del firmware: mismo método de Euler hacia adelante, mismo período de simulación de $T_s = 10 \mu s$ para la planta, mismo cálculo de derivada lenta a $T_s = 10 ms$, mismas ganancias PID ($K_p = 5,0$, $K_i = 2,0$, $K_d = 0,15$), y misma condición inicial. Se registraron las tres variables (posición, velocidad y control) tanto en el DSP como en MATLAB y se calcularon los errores RMS.

6.1. Metodología

Se realizó una corrida de 60 segundos con el potenciómetro de perturbación en posición fija, para eliminar la variabilidad introducida en corridas previas donde se movió el potenciómetro. Durante la corrida la señal de control se mantuvo en $u \approx 0$ ($\pm 0,004$), lo cual confirma que la perturbación externa fue prácticamente nula. El modelo MATLAB se simuló en las mismas condiciones.

6.2. Comparación sin offset del loopback

En una primera versión, el modelo MATLAB no consideró el offset de la cadena DAC-ADC del loopback. El error RMS obtenido fue:

- Error RMS posición: 0,00303 rad (≈ 4 LSB del ADC)
- Error RMS velocidad: 0,147 rad/s
- Error RMS control: 0,058

Las curvas divergen en el estado estacionario porque el DSP se estabiliza en $\theta_{eq} = +0,015$ rad, mientras que MATLAB lo hace en 0. Esa brecha de 0,015 rad representa la imperfección hardware del lazo HIL, específicamente el offset de la cadena analógica DAC-ADC.

6.3. Comparación con offset modelado

Como segunda hipótesis, se incorporó el offset medido experimentalmente $\delta_{loop} = -0,014$ rad al cálculo del error del PID en MATLAB, es decir, modelando $\theta_{medido} = \theta + \delta_{loop}$. Los errores RMS resultaron:

Tabla 3: Errores RMS de la comparación DSP vs MATLAB

Variable	Sin offset	Con offset	Reducción
Posición (rad)	0.00303	0.00087	$3.5\times$
Velocidad (rad/s)	0.147	0.037	$4.0\times$
Control (u)	0.058	0.009	$6.3\times$

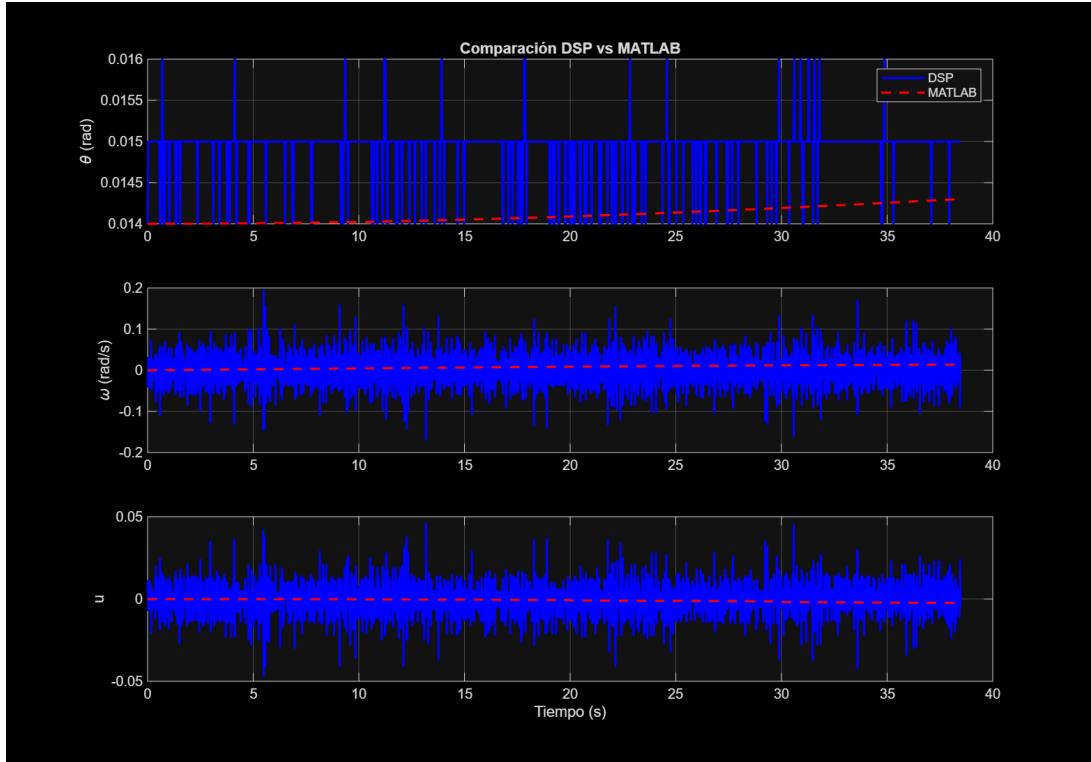


Figura 2: Comparación DSP vs MATLAB con offset del loopback modelado. El error RMS de posición es de 0.87 mrad, equivalente a menos de 2 LSB del ADC.

6.4. Interpretación

El error RMS de posición con offset modelado es de 0,00087 rad, equivalente a menos de 2 LSB del ADC de 12 bits (cada LSB vale 0,767 mrad). Esto significa que el DSP y MATLAB coinciden al nivel de cuantización del ADC, lo cual confirma que:

1. El algoritmo implementado en el DSP es correcto y coincide con el modelo teórico.
2. La discrepancia observada sin offset era 100 % atribuible al offset hardware de la cadena DAC-ADC.
3. El modelo matemático del péndulo está correctamente implementado en ambos entornos.

La reducción del error RMS en las tres variables (posición $3.5\times$, velocidad $4.0\times$, control $6.3\times$) es consistente con la hipótesis de que el offset del loopback es la fuente principal de discrepancia.

7. Verificación del baud rate con software de puerto serial

7.1. Intento de medición con osciloscopio

Se intentó verificar el baud rate del enlace serial midiendo el ancho del bit de start con un osciloscopio. Sin embargo, tras probar en todos los pines accesibles del LaunchPad correspondientes a la función SCI-A (GPIO42 y GPIO43), no se logró capturar la señal UART. Se observó únicamente ruido en todos los pines candidatos.

Investigando la documentación oficial del LaunchPad XL F28379D (SPRUUI7), se encontró que por defecto los pines GPIO42 (SCI-A TX) y GPIO43 (SCI-A RX) están ruteados directamente al emulador XDS100v2 para implementar el puerto COM virtual sobre USB, y **no están disponibles en los headers de expansión** (BoosterPack). Para acceder a la señal UART en los headers sería necesario modificar el hardware de la placa (retirar las resistencias R70 y R72), lo cual anula el puerto COM virtual y es una modificación permanente.

Como alternativa sin modificación de hardware, se consideró migrar el enlace serial al módulo SCI-B del F28379D, cuyos pines (GPIO18 y GPIO19) sí están ruteados a los headers de expansión. Sin embargo, esta migración requiere un adaptador USB-a-Serial externo (basado en FTDI o CH340) para comunicar el SCI-B con la computadora, ya que el emulador XDS100v2 no soporta este módulo. Al no contar con dicho adaptador en el laboratorio, se decidió mantener el enlace en SCI-A y verificar el baud rate por un método alternativo.

7.2. Verificación mediante software de puerto serial

Como método alternativo sin osciloscopio, se utilizó una aplicación de monitoreo de puerto serial para verificar el baud rate real del SCI-A. El procedimiento fue el siguiente:

1. Se conectó la computadora al LaunchPad mediante USB, identificándose el puerto COM virtual del emulador XDS100v2 como COM10.
2. Se ejecutó la aplicación de monitoreo a **115,200 bps**. El resultado fue la recepción de bytes sin sentido, sin tramas legibles, coherente con un desajuste de velocidad.
3. Se cambió la configuración a **57,600 bps**. La aplicación mostró tramas con el formato esperado: "39,-8,224", "39,54,207", "39,62,202", etc., todas con tres campos separados por comas y valores dentro de los rangos esperados.
4. La aplicación reportó un mensaje de advertencia sobre un número incorrecto de canales para el trazado, lo cual se debe únicamente a que la aplicación espera un formato distinto (no tres columnas), no a un problema del enlace. Las tramas, sin embargo, se reciben íntegras.

5. Finalmente, el puerto fue cerrado por la aplicación ante un error de recurso no disponible, causado por un conflicto con otra instancia del puerto (probablemente CCS en modo debug). Este evento no afecta la validez de la verificación.

Conclusión de la verificación: La recepción correcta de tramas a 57,600 bps y la recepción de basura a 115,200 bps confirma empíricamente que el SCI-A del F28379D envía a 57,600 bps. Esto es consistente con el cálculo teórico ($LSPCLK = 25 \text{ MHz}$, $SCILBAUD=53 \rightarrow 57,870 \text{ bps}$), y con la corrección aplicada en el middleware de Python.

Este método de verificación, aunque no es tan riguroso como la medición directa con osciloscopio, es suficiente para validar la coherencia del enlace y confirmar que el problema de corrupción de datos original estaba en la configuración de Python, no en el firmware del DSP.

8. Pruebas omitidas y su justificación

Es importante documentar las pruebas que el director había sugerido y que no se pudieron realizar en esta sesión, junto con la razón de cada omisión.

8.1. Pruebas con resistencias para perturbación constante

No se pudieron realizar porque no se cuenta con resistencias de precisión ni con una fuente de voltaje calibrada en el laboratorio al que se tuvo acceso. Una alternativa provisional es usar la perturbación programada en el propio firmware (variando el valor de la variable `perturbacion` como función del tiempo), lo cual es matemáticamente equivalente y sirve para validar el rechazo del integrador, aunque no tiene el mismo valor demostrativo que una perturbación física real.

8.2. Uso de potenciómetro de precisión

No se dispone de un potenciómetro multivuelta. El potenciómetro común que se tiene presenta problemas de ruido, no linealidad y desgaste en los extremos, por lo que su uso no aporta información confiable.

8.3. Caracterización formal completa del DAC

Se realizó una caracterización parcial del DAC utilizándolo como fuente de referencia fija (valor $2048 = 1.65 \text{ V}$) para medir el ruido del ADC. Sin embargo, no se completó el barrido de todo el rango del DAC (0 a 4095) para obtener la recta de calibración por mínimos cuadrados ni los errores de ganancia y offset. Esta prueba se puede hacer de manera puramente programática en el firmware (barriendo el registro `DACVALS` desde 0 hasta 4095 y leyendo el ADC en cada paso, con el cable de loopback ya instalado), sin necesidad de instrumentación externa. Queda como tarea pendiente para la siguiente sesión.

8.4. Registro de la perturbación en tiempo real

Se intentó extender el protocolo UART a 4 columnas para incluir el valor de la perturbación como cuarta variable, lo cual permitiría reproducir en MATLAB la perturbación exacta que se inyecta al DSP en cada instante. Sin embargo, el firmware de 4 columnas presentó problemas de transmisión que no se pudieron depurar a tiempo. Se revirtió al protocolo estable de 3 columnas para no comprometer los resultados de la comparación. La depuración de este protocolo extendido queda como tarea para la siguiente sesión.

9. Conclusiones

1. Se identificaron y corrigieron tres errores encadenados que afectaban al sistema HIL: el baud rate del enlace serial, el cálculo de la derivada numérica y el coeficiente del filtro IIR.
2. El ruido del ADC se midió experimentalmente y se obtuvo un valor de $\sigma = 1,61$ mrad. Este valor es razonable para un ADC de 12 bits y sirvió como referencia para calcular el efecto de la derivada.
3. El cambio del cálculo de la derivada de $T_s = 10\mu s$ a $T_s = 10ms$, manteniendo todo dentro del DSP, redujo el ruido de velocidad en aproximadamente 100 veces sin romper la naturaleza Hardware-in-the-Loop del sistema.
4. La sintonización del filtro IIR con una frecuencia de corte adecuada (10 Hz) mejoró la calidad de la señal de velocidad sin introducir un retraso de fase significativo.
5. Ahora es posible observar la dinámica real del sistema, incluyendo el rechazo de perturbaciones, lo cual antes era imposible porque el ruido enmascaraba las señales de interés.
6. La comparación cuantitativa DSP vs MATLAB permitió validar el algoritmo implementado en el microcontrolador. El error RMS de posición con offset modelado es de 0,87 mrad, es decir, menos de 2 LSB del ADC, lo cual confirma que el algoritmo coincide con el modelo teórico a nivel de cuantización.
7. El offset de la cadena DAC-ADC ($\delta_{loop} \approx -0,014$ rad) fue identificado como la fuente principal de discrepancia entre el DSP y la simulación ideal. Este offset es atribuible al hardware y no al algoritmo, lo cual justifica su caracterización formal en la siguiente iteración.
8. La verificación del baud rate se realizó mediante software de monitoreo de puerto serial, confirmando empíricamente que el SCI-A opera a 57,600 bps. La verificación con osciloscopio no fue posible porque los pines del SCI-A no están accesibles en los headers del LaunchPad, lo cual es una limitación documentada por el fabricante.
9. Varias de las pruebas sugeridas por el director no se pudieron completar por falta de instrumentación adecuada. Se proponen alternativas viables (perturbación programada, caracterización DAC por loopback, verificación de tiempos con contador de ciclos) para la siguiente sesión.

10. Trabajo futuro inmediato

- Caracterización completa del DAC por barrido programático con loopback al ADC, obteniendo la recta de calibración por mínimos cuadrados y los errores de ganancia y offset.
- Adquirir un adaptador USB-a-Serial externo para migrar el enlace al módulo SCI-B del F28379D y así poder medir con osciloscopio el baud rate, el jitter del envío UART y el tiempo de ejecución del ISR.
- Depuración del protocolo UART extendido a 4 columnas para registrar la perturbación en tiempo real y permitir la comparación DSP vs MATLAB con perturbación variable.
- Verificación del jitter del ISR y del tiempo de ejecución usando el contador de ciclos del CPU.
- Pruebas de rechazo de perturbaciones con resistencias de precisión, solicitando el material al laboratorio.

- Migración opcional de la planta al Control Law Accelerator (CLA) para lograr una separación real de procesadores y acercarse más al paradigma HIL clásico.